AIF-C01 · 內容領域 3

基礎模型的應用

3.1 FM 應用的設計考量 · 3.2 提示詞工程技術
3.3 FM 訓練與微調 · 3.4 評估 FM 效能

28%

考試計分占比

依 AWS 官方考試指南（AIF-C01）重新整理

領域 3 總覽：任務陳述

3.1

描述使用 FM 的應用程式設計考量

FM 選擇準則 · 推論參數 · 客製化成本權衡 · RAG 與知識庫 · 向量資料庫 · AI 代理

3.2

選擇有效的提示詞工程技術

提示詞結構 · 零 / 單一 / 少樣本與思路鏈 · 最佳實務 · 風險（注入、越獄）· Prompt Management

3.3

描述 FM 的訓練和微調流程

預訓練、微調、持續預訓練、蒸餾 · 微調方法 · 資料準備與 RLHF

3.4

描述評估 FM 效能的方法

人工評估、基準資料集、Bedrock 評估 · ROUGE、BLEU、BERTScore、LLM 評審 · 業務指標

選擇 FM 的準則

成本
Token 單價、託管與客製化成本

模態
文字、圖像、音訊、影片、多模態

延遲
即時互動需要快速回應的模型

多語言支援
是否支援目標語言，例如繁體中文

模型大小
參數越多通常越強，但越慢越貴

模型複雜度
任務難度是否需要強推理能力

自定義能力
是否支援微調、持續預訓練、蒸餾

輸入 / 輸出長度
上下文視窗能放多少內容、最多回多少字

提示詞快取 Prompt caching
重複使用的長前綴可快取，降低成本與延遲

推論參數如何影響回應

溫度 Temperature

控制隨機性

- 低（如 0–0.3）：穩定、保守，適合事實問答、分類
- 高（如 0.8–1）：多樣、有創意，適合腦力激盪、文案

Top-p / Top-k

限制候選詞範圍

- Top-k：只從機率最高的 k 個詞挑選
- Top-p：只從累積機率達 p 的詞挑選；值越小越保守

輸入 / 輸出長度

最大 Token 數

- 限制最大輸出長度可控制成本與回應時間
- 設太短會截斷回答；輸入過長可能超出上下文視窗

停止序列 Stop sequence

指定停止點

- 遇到特定字串就停止生成
- 可控制輸出格式，避免多餘內容

FM 客製化方式的成本權衡

方式	做法	成本	適用情境
情境學習 In-context	在提示詞中放指令與範例，不改模型	最低	快速原型、通用任務
RAG	檢索外部知識放進提示詞	中	知識常更新、需引用來源
模型蒸餾 Distillation	用大模型（教師）的輸出訓練小模型（學生）	中	要接近大模型品質，但更快更便宜
微調 Fine-tuning	以標記資料調整模型權重	高	特定風格、格式、領域任務
持續預訓練	以大量未標記領域資料繼續預訓練	更高	專業領域語言（醫療、法律）
預訓練 Pre-training	從頭訓練新模型	最高	極少數需完全自有模型的情況

RAG 與 Amazon Bedrock 知識庫



RAG 的商業價值

- 使用最新資訊，不必重新訓練
- 以文件為依據，減少幻覺
- 可附上來源，回答可驗證
- 整合企業私有資料，成本低於微調

Amazon Bedrock 知識庫

- 連接 S3 等資料來源
- 自動分塊、嵌入與建立索引
- 儲存到向量資料庫並語意檢索
- 與 Bedrock 模型整合，可做 RAG 評估

在向量資料庫中儲存嵌入的 AWS 服務



Amazon OpenSearch Service

搜尋與分析引擎，支援 k-NN 向量搜尋；Bedrock 知識庫最常用的向量儲存（含 Serverless）



Amazon Aurora

Aurora PostgreSQL 搭配 pgvector 擴充功能，在關聯式資料庫中存向量



Amazon Neptune

圖形資料庫；Neptune Analytics 支援向量搜尋，可做 GraphRAG



Amazon RDS for PostgreSQL

同樣透過 pgvector 支援向量儲存與相似度搜尋

AI 代理的角色與商業應用

AI 代理能理解目標、規劃步驟、呼叫工具與 API、根據結果調整，自主完成多步驟任務——不只是「回答」，而是「做事」



客服自動化

查詢訂單、處理退貨、更新資料



預約與流程處理

查行事曆、預訂、發送確認



文件與報告

蒐集資料、分析並產出報告



軟體開發

撰寫、測試與修正程式（Kiro）



資料分析

查詢資料庫並整理洞察



IT 營運

監控告警、自動排除故障

提示詞的結構

上下文 Context	背景與角色：「你是一位 AWS 解決方案架構師，客戶是零售業……」
指令 Instruction	明確要做的事與格式：「用表格列出 3 個建議，每項不超過 30 字」
輸入資料 Input	要處理的內容：客戶需求、文件或問題
負面提示 Negative prompt	不要什麼：「不要提到價格」「圖片中不要出現文字」→ 就是限制

提示詞工程技術

零樣本 Zero-shot

不給範例，直接下指令。適合簡單、通用任務

單一樣本 One-shot

給 1 個範例示範格式

少樣本 Few-shot

給 2–5 個範例，讓模型學到模式；適合分類與固定格式

思路鏈 Chain-of-Thought

要求逐步推理（「請一步一步思考」）；適合數學與邏輯

提示範本 Template

把固定結構做成範本，以變數填入內容，確保一致

提示詞工程的好處與最佳實務

回應品質改進
清楚的提示詞大幅提升準確與相關性

實驗
嘗試不同寫法並比較結果

防護機制
在提示詞中設定界線與拒答規則

發現
透過測試找出模型的能力與限制

特異性和簡潔性
具體明確、去除不必要的文字

使用多評論
多次生成或多角度審查後再定稿

實用口訣：設定角色 → 給上下文 → 下明確指令 → 指定格式與長度 → 提供範例 → 迭代測試

提示詞工程的風險與限制



曝險 Exposure

提示詞或回應洩露敏感資料、系統提示詞或內部資訊



中毒 Poisoning

在訓練資料或 RAG 文件中植入惡意內容，影響模型輸出



劫持 Hijacking

以提示注入覆蓋原本指令，讓模型做出非預期行為



越獄 Jailbreak

用角色扮演等技巧繞過安全限制，誘導產生有害內容

防範：Bedrock 防護機制（提示攻擊過濾）、輸入驗證、最小權限、輸出過濾、人工審核

Amazon Bedrock Prompt Management



集中儲存提示詞

把提示詞存成可重用資源，不寫死在程式碼中



版本控制

每次修改建立版本，可比較與回溯



變數與範本

以變數填入不同內容，同一範本多處使用



測試與比較

在不同模型與參數下測試同一提示詞



部署到應用程式

應用程式直接引用指定版本，更新不必改程式

訓練 FM 的關鍵要素

1

預先訓練 Pre-training

以海量未標記資料學習通用知識；成本極高，通常由模型供應商完成

2

微調 Fine-tuning

以較少量的標記資料調整權重，讓模型擅長特定任務

3

持續預先訓練 Continued pre-training

以大量未標記的領域資料繼續預訓練，學會專業領域語言

4

蒸餾 Distillation

用大模型的輸出訓練小模型，保留品質、降低成本與延遲

分辨關鍵：微調用「標記資料」；持續預訓練用「未標記資料」

微調 FM 的方法

指令調校 Instruction tuning

以「指令 → 理想回應」配對資料訓練，讓模型更會遵循指令

適應特定領域

以醫療、法律、金融等領域資料調整，學會專業術語與寫法

轉移學習 Transfer learning

把預訓練學到的知識轉移到新任務，只需少量資料

持續預先訓練

以未標記的領域資料延續預訓練，擴充領域知識

如何準備微調資料

資料策管 Curation

挑選高品質、相關、去重、無錯誤的樣本

管控 Governance

確認資料來源、授權、隱私與存取權限

大小 Size

資料量要足夠；品質比數量更重要

標記 Labeling

提供正確的輸入與期望輸出配對

代表性 Representativeness

涵蓋各種情境與族群，避免偏差

RLHF 人類回饋強化學習

由人類對回應排序，訓練模型更符合人類偏好

RLHF 流程：模型產生多個回應 → 人類評分排序 → 訓練獎勵模型 → 以強化學習優化 FM（SageMaker Ground Truth 可蒐集人類回饋）

評估 FM 效能的方法



真人參與評估 Human-in-the-loop

由人類評分關聯性、風格、品牌語氣等主觀品質



基準測試資料集 Benchmark

以標準資料集比較不同模型在相同任務上的表現



Amazon Bedrock 評估

自動（程式化）評估、人工評估、LLM 評審、RAG 評估

程式化評估

準確度、穩健性、毒性等演算法指標

人工評估

自有團隊或 AWS 管理的評審

LLM 評審

以 LLM 評正確性、完整性、有害性

RAG 評估

檢索相關性與覆蓋率；生成忠實度

評估 FM 的指標

ROUGE	*Recall-Oriented Understudy for Gisting Evaluation*	比較生成摘要與參考摘要的重疊字詞，重視召回	摘要
BLEU	*Bilingual Evaluation Understudy*	比較生成譯文與參考譯文的 n-gram 重疊，重視精確	翻譯
BERTScore	*以 BERT 嵌入計算語意相似度*	看「意思」是否相近，而非逐字比對	語意相似
LLM 評審 LLM-as-a-judge	*用另一個 LLM 依評分標準打分*	可評正確性、完整性、語氣、有害性	開放式回答

FM 應用的效能評估與業務對齊

RAG 應用

- 檢索：上下文相關性、覆蓋率
- 生成：忠實度（是否根據文件）、正確性

代理應用

- 任務是否完成、工具是否選對
- 步驟數、錯誤與重試次數

工作流

- 端到端成功率
- 各步驟延遲與成本

對齊業務目標的指標

任務完成率

使用者的目標有沒有達成

使用者滿意度

評分、回饋、留存

每次互動成本

Token 與基礎設施成本

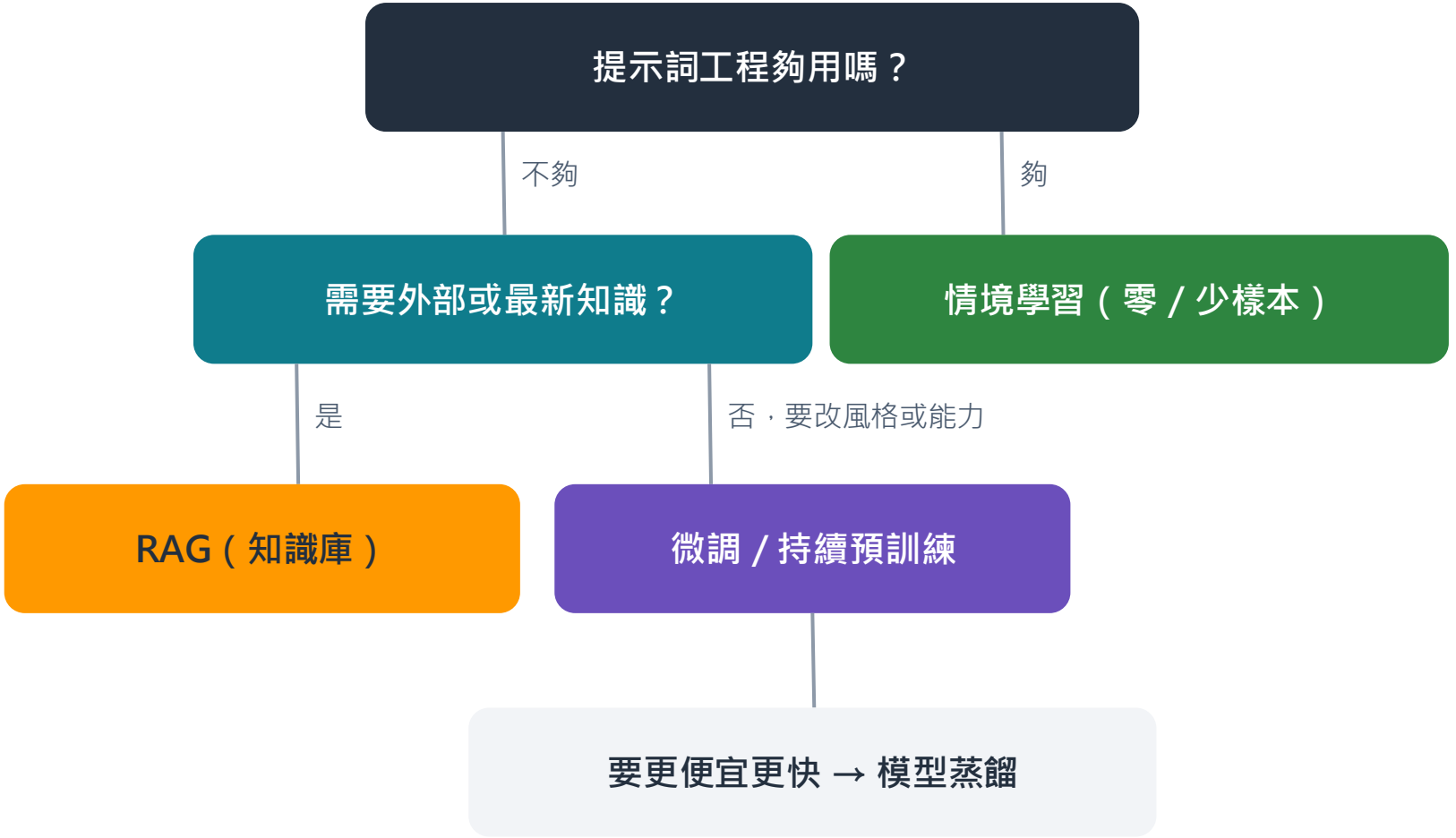
生產力

節省的時間與人力

使用者參與度

使用頻率與互動深度

決策樹：該用哪種客製化


提示詞工程夠用嗎？
不夠
夠
需要外部或最新知識？
情境學習（零 / 少樣本）
是
否，要改風格或能力
RAG（知識庫）
微調 / 持續預訓練
要更便宜更快 → 模型蒸餾

情境練習

1. 大量請求共用同一段長系統提示詞 [redacted]

2. 事實問答要穩定、減少隨機性 [redacted]

3. 要大模型品質，但推論更便宜 [redacted]

4. 客服要依每週更新的手冊回答 [redacted]

5. Bedrock 知識庫最常用的向量儲存 [redacted]

6. 給 3 個範例讓格式一致 [redacted]

7. 使用者試圖用指令覆蓋系統提示 [redacted]

8. 提示詞要版本控制並可回溯 [redacted]

9. 評估自動摘要的品質 [redacted]

10. 評估機器翻譯的品質 [redacted]

情境練習

1. 大量請求共用同一段長系統提示詞	**提示詞快取**
2. 事實問答要穩定、減少隨機性	**降低 Temperature**
3. 要大模型品質，但推論更便宜	**模型蒸餾**
4. 客服要依每週更新的手冊回答	**RAG / 知識庫**
5. Bedrock 知識庫最常用的向量儲存	**OpenSearch Service**
6. 給 3 個範例讓格式一致	**少樣本 Few-shot**
7. 使用者試圖用指令覆蓋系統提示	**提示注入 / 劫持**
8. 提示詞要版本控制並可回溯	**Prompt Management**
9. 評估自動摘要的品質	**ROUGE**
10. 評估機器翻譯的品質	**BLEU**

自我檢核：領域 3 官方目標

☑	3.1 **FM 選擇準則與推論參數**	成本、模態、延遲、多語、大小、長度、提示詞快取；溫度
☑	3.1 **客製化成本權衡**	預訓練、微調、情境學習、RAG、蒸餾
☑	3.1 **RAG、向量資料庫與 AI 代理**	Bedrock 知識庫；OpenSearch、Aurora、Neptune、RDS PostgreSQL
☑	3.2 **提示詞結構與技術**	上下文、指令、負面提示；零 / 單一 / 少樣本、思路鏈、範本
☑	3.2 **最佳實務與風險**	特異性、實驗、防護；曝險、中毒、劫持、越獄
☑	3.2 **Prompt Management**	版本控制與管理策略
☑	3.3 **訓練要素與微調方法**	預訓練、微調、持續預訓練、蒸餾；指令調校、領域適應、轉移學習
☑	3.3 **微調資料準備**	策管、管控、大小、標記、代表性、RLHF
☑	3.4 **FM 評估方法與指標**	人工、基準、Bedrock 評估；ROUGE、BLEU、BERTScore、LLM 評審
☑	3.4 **應用評估與業務對齊**	RAG、代理、工作流；任務完成率、滿意度、每次互動成本